

Object Oriented Programming Lab

Lab Tasks: Structures in C++

Instructions:

- Attempt all six questions. Each question includes a hint — use it only if you get stuck.
- Write, compile, and test each program in a separate .cpp file.
- Every dynamically allocated block must be correctly deallocated.
- Solutions are provided at the end of this document for self-checking after you attempt each question.

Easy Questions

Question 1 [EASY]

Define a Book structure (title, author, price). Read a book's details from the user, then write a function that takes the structure BY VALUE and displays all of its details.

Hint: Read each member with cin (e.g. cin >> b.title >> b.author >> b.price), then declare your display function as void displayBook(Book b) — passing by value is fine here since the function only needs to READ the data, matching Topic 6 of your lab manual.

Question 2 [EASY]

Define an Item structure (name, quantity, price). Create an array of at least 3 items, then write a function that computes and returns the TOTAL COST of the whole array (sum of quantity x price for every item).

Hint: Pass the array as const Item items[] (or const Item items) together with its size — const tells the compiler (and anyone reading your code) that the function only reads the array and never modifies it, exactly like Topic 7's const-reference pattern.*

Medium Questions

Question 3 [MEDIUM]

Define an Employee structure (name, salary). Write a function that gives EVERY employee in an array a percentage raise, modifying the original array directly (the change must be visible in main() after the function returns).

Hint: Since the function must modify the CALLER's original array, pass it by pointer or reference — not by value. A parameter like Employee staff (or Employee arr[]) already*

gives you access to the original memory; just loop through and update `staff[i].salary` directly, similar to `raiseSalary()` in Topic 7 of your lab manual.

Question 4 [MEDIUM]

Define a Point structure (x, y). Write TWO functions: one that takes two points BY VALUE and returns the distance between them (as a double), and another that takes two points BY VALUE and RETURNS a brand-new Point structure representing their midpoint.

Hint: The distance formula is $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$ — use `<cmath>`'s `sqrt()`. For the midpoint function, build a local Point variable, set its x and y to the averages of the two input points' coordinates, and return it by value — this is exactly the 'builder function' pattern from Topic 8 of your lab manual.

Hard Questions

Question 5 [HARD]

Define an Address structure (city, street) and an Employee structure that contains an Address as a nested member. Create an array of employees, then write a function that searches for and returns a POINTER to the first employee living in a given city (or nullptr if none match). Using the returned pointer, update that employee's street name and show the change reflected in the original array.

Hint: Access the nested member with `arr[i].address.city` inside your search loop (Topic 4's nested-structure pattern). The function should return `&arr[i]` the moment it finds a match — this is a pointer INTO the array, not new memory, so the caller must never delete it (Topic 8's 'safe pattern 2': returning a pointer to memory the caller already owns).

Question 6 [HARD]

Design a 'Student Directory' that stores an array of POINTERS to dynamically allocated Student structures (name, roll number, GPA). The directory must automatically grow (double its capacity) when full, support adding a student (returning a pointer to the newly created record), support searching by roll number, and correctly free every dynamically allocated Student plus the pointer array itself when the program ends.

Hint: This combines several topics: the directory itself is a `Student**` (an array of pointers to structures, from Topic 5), each new student is created with new `Student{...}` and its pointer stored in the array. When growing, allocate a new, larger `Student**` array, copy the existing POINTERS across (not the actual structure data), then free only the OLD pointer array — the Student records themselves must survive. Your `addStudent()` function returns a pointer to the structure it just created, which is Topic 8's 'return a pointer to dynamically allocated memory' pattern.

Solutions

The solutions below are one possible correct approach for each question. Compare your own logic against these after attempting the problems yourself — there may be other valid ways to solve the same task.

Solution 1: Reading and Displaying a Book Structure

```
#include <iostream>
#include <string>
using namespace std;

struct Book {
    string title;
    string author;
    double price;
};

// Pass by value -- displays the book (does not need to modify it)
void displayBook(Book b) {
    cout << "Title : " << b.title << endl;
    cout << "Author: " << b.author << endl;
    cout << "Price : " << b.price << endl;
}

int main() {
    Book b1;
    cout << "Enter title, author, price: ";
    cin >> b1.title >> b1.author >> b1.price;

    displayBook(b1);
    return 0;
}
```

Expected Output:

```
Enter title, author, price: Title : CppPrimer
Author: StanleyLippman
Price : 59.99
```

Solution 2: Total Cost of an Array of Item Structures

```
#include <iostream>
#include <string>
using namespace std;

struct Item {
    string name;
    int quantity;
    double price;
};

// const reference -- efficient (no copy) and guarantees the array isn't modified
double totalCost(const Item items[], int size) {
    double total = 0;
    for (int i = 0; i < size; i++)
```

```

        total += items[i].quantity * items[i].price;
    return total;
}

int main() {
    Item cart[3] = {
        { "Notebook", 5, 2.50 },
        { "Pen",      10, 0.75 },
        { "Eraser",   3, 0.30 }
    };

    cout << "Total cost of items in cart: " << totalCost(cart, 3) << endl;
    return 0;
}

```

Expected Output:

```
Total cost of items in cart: 20.9
```

Solution 3: Applying a Raise to an Array of Employees (By Reference)

```

#include <iostream>
#include <string>
using namespace std;

struct Employee {
    string name;
    double salary;
};

// Modifies the ORIGINAL array's salaries directly through pointer access
void applyRaise(Employee* staff, int size, double percent) {
    for (int i = 0; i < size; i++)
        staff[i].salary += staff[i].salary * (percent / 100.0);
}

void printStaff(const Employee* staff, int size) {
    for (int i = 0; i < size; i++)
        cout << " " << staff[i].name << ": " << staff[i].salary << endl;
}

int main() {
    Employee staff[3] = {
        { "Bilal", 45000 },
        { "Sara",  60000 },
        { "Umar",  52000 }
    };

    cout << "Before raise:\n";
    printStaff(staff, 3);

    applyRaise(staff, 3, 8);    // 8% raise for everyone

    cout << "\nAfter 8% raise:\n";
    printStaff(staff, 3);

    return 0;
}

```

Expected Output:

```
Before raise:
  Bilal: 45000
  Sara: 60000
  Umar: 52000

After 8% raise:
  Bilal: 48600
  Sara: 64800
  Umar: 56160
```

Solution 4: Distance and Midpoint Between Two Points

```
#include <iostream>
#include <cmath>
using namespace std;

struct Point {
    double x;
    double y;
};

// Takes two points by value, returns a computed distance
double distanceBetween(Point p1, Point p2) {
    double dx = p2.x - p1.x;
    double dy = p2.y - p1.y;
    return sqrt(dx * dx + dy * dy);
}

// Builds and returns an entire NEW Point structure by value
Point midpoint(Point p1, Point p2) {
    Point mid;
    mid.x = (p1.x + p2.x) / 2.0;
    mid.y = (p1.y + p2.y) / 2.0;
    return mid;
}

int main() {
    Point a = { 2.0, 3.0 };
    Point b = { 8.0, 11.0 };

    cout << "Distance between A and B: " << distanceBetween(a, b) << endl;

    Point m = midpoint(a, b);
    cout << "Midpoint: (" << m.x << ", " << m.y << ")" << endl;

    return 0;
}
```

Expected Output:

```
Distance between A and B: 10
Midpoint: (5, 7)
```

Solution 5: Searching Nested Structures and Returning a Pointer

```

#include <iostream>
#include <string>
using namespace std;

struct Address {
    string city;
    string street;
};

struct Employee {
    string name;
    Address address;    // nested structure
};

// Searches the array and returns a POINTER to the first employee living in
// the given city, or nullptr if none is found. No new memory is created here,
// so the caller must NOT delete the returned pointer.
Employee* findByCity(Employee arr[], int size, const string& city) {
    for (int i = 0; i < size; i++)
        if (arr[i].address.city == city)
            return &arr[i];
    return nullptr;
}

int main() {
    Employee staff[4] = {
        { "Ahmed", { "Lahore", "Mall Road" } },
        { "Sana", { "Peshawar", "GT Road" } },
        { "Bilal", { "Peshawar", "University Road" } },
        { "Iqra", { "Karachi", "Shahrah-e-Faisal" } }
    };

    Employee* found = findByCity(staff, 4, "Peshawar");

    if (found != nullptr) {
        cout << "Found employee in Peshawar: " << found->name
              << ", Street: " << found->address.street << endl;

        // Modifying through the returned pointer affects the original array
        found->address.street = "Updated Street";
        cout << "After update, staff[1].address.street = " << staff[1].address.street <<
endl;
    } else {
        cout << "No employee found in that city." << endl;
    }

    return 0;
}

```

Expected Output:

```

Found employee in Peshawar: Sana, Street: GT Road
After update, staff[1].address.street = Updated Street

```

Solution 6: Growable Student Directory (Array of Pointers to Structures)

```

#include <iostream>
#include <string>

```

```

using namespace std;

struct Student {
    string name;
    int rollNo;
    double gpa;
};

class StudentDirectory {
private:
    Student** students; // array of pointers to dynamically allocated Students
    int capacity;
    int count;

    void grow() {
        int newCapacity = (capacity == 0) ? 2 : capacity * 2;
        Student** newArray = new Student*[newCapacity];
        for (int i = 0; i < count; i++)
            newArray[i] = students[i]; // copy the POINTERS only
        delete[] students; // free only the OLD pointer array
        students = newArray;
        capacity = newCapacity;
    }

public:
    StudentDirectory() : students(nullptr), capacity(0), count(0) {}

    ~StudentDirectory() {
        for (int i = 0; i < count; i++)
            delete students[i]; // free each dynamically allocated Student
        delete[] students; // free the pointer array itself
    }

    // Returns a POINTER to a newly created, dynamically allocated Student --
    // demonstrates "returning a pointer from a function" as well.
    Student* addStudent(const string& name, int rollNo, double gpa) {
        if (count == capacity) grow();
        Student* s = new Student{ name, rollNo, gpa };
        students[count++] = s;
        return s;
    }

    Student* findByRollNo(int rollNo) {
        for (int i = 0; i < count; i++)
            if (students[i]->rollNo == rollNo)
                return students[i];
        return nullptr;
    }

    void printAll() const {
        for (int i = 0; i < count; i++)
            cout << " " << students[i]->rollNo << " " << students[i]->name
                << " GPA: " << students[i]->gpa << endl;
    }
};

int main() {
    StudentDirectory dir;

```

```
dir.addStudent("Hassan", 301, 3.4);
dir.addStudent("Rabia", 302, 3.9);
dir.addStudent("Fahad", 303, 2.8);
dir.addStudent("Sadiah", 304, 3.6); // triggers a resize

cout << "All students:\n";
dir.printAll();

cout << "\nSearching for roll number 303...\n";
Student* found = dir.findByRollNo(303);
if (found != nullptr)
    cout << "Found: " << found->name << ", GPA: " << found->gpa << endl;

return 0; // destructor frees every Student and the pointer array
}
```

Expected Output:

```
All students:
301 Hassan GPA: 3.4
302 Rabia GPA: 3.9
303 Fahad GPA: 2.8
304 Sadia GPA: 3.6
```

```
Searching for roll number 303...
Found: Fahad, GPA: 2.8
```